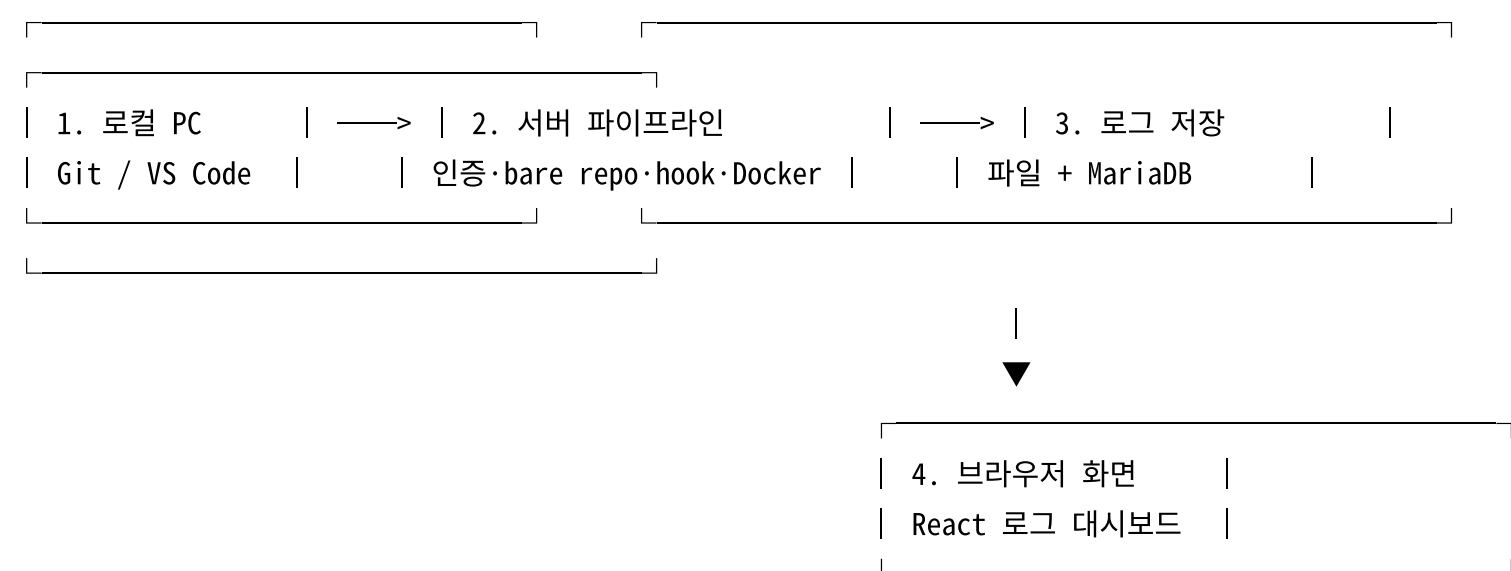


전체 흐름 아키텍처

운영 경로 /app/logplus-platform

로컬 PC의 Git 요청이 서버 빌드·로그 저장을 거쳐

LogPlus 브라우저 화면에 표시되는 전체 흐름을 정리한다.



Git 원본은 /repos, 빌드 작업본은 projects, 로그 본문은 logs, 조회 기준과 분석 이력은 MariaDB에 분리된다.

웹	React/Vite	5174
	FastAPI	8000
	Git Smart HTTP	8080

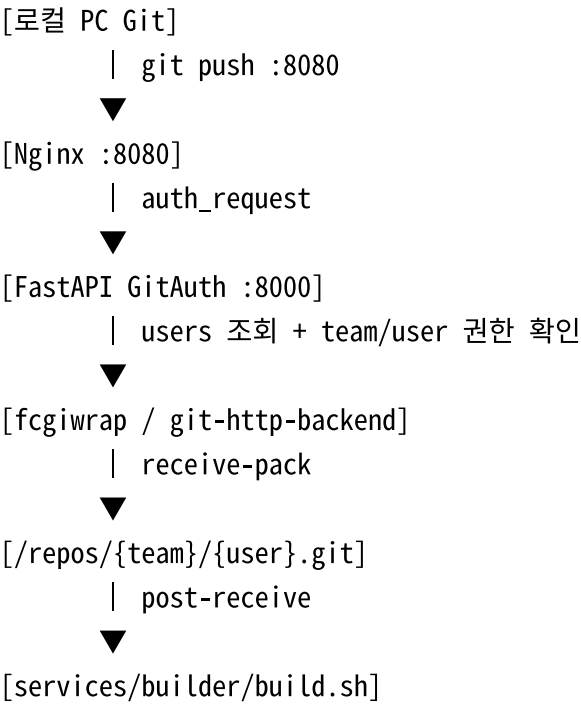
현재 화면은 사용자 Docker 컨테이너의 웹 화면이 아니라, Docker 실행 결과인 LIB/RUN 로그와 AI 분석 결과를 표시하는 LogPlus 로그 대시보드 화면이다.

- 1쪽 전체 개요
- 2쪽 컴포넌트 연결
- 3쪽 요청·데이터 흐름
- 4쪽 저장 경계·검증·다음 작업

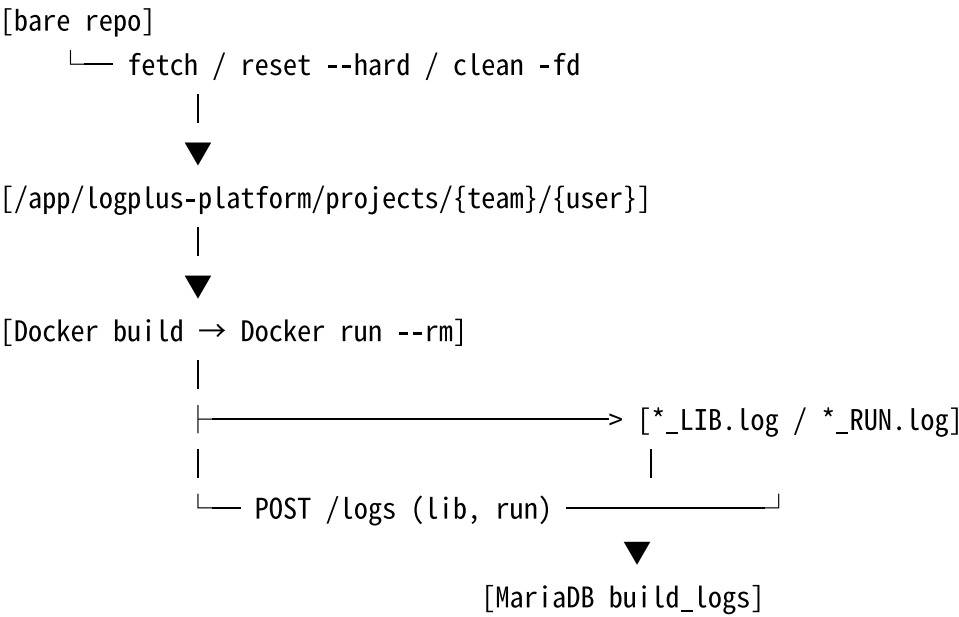
LOGPLUS COMPONENT MAP

전체 컴포넌트 연결

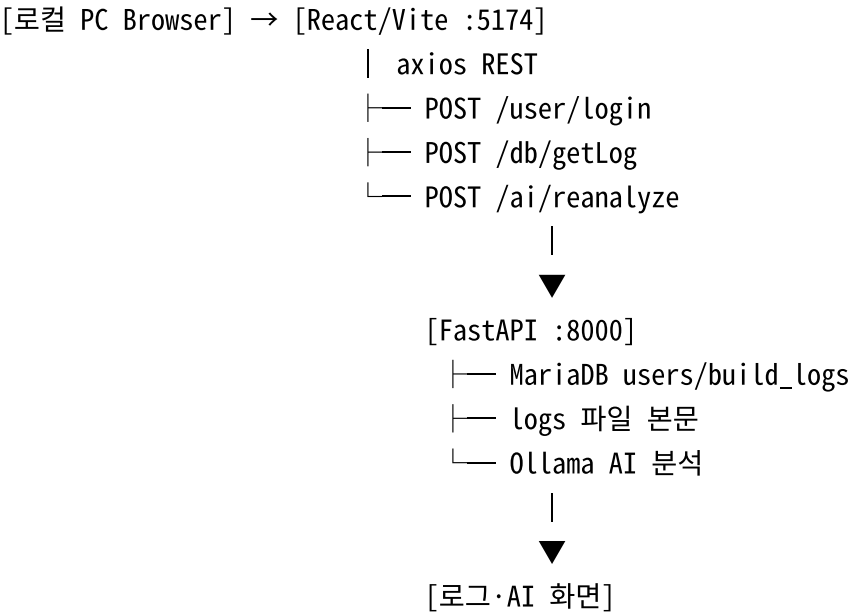
GIT PATH



BUILD PATH



WEB PATH



컴포넌트 책임

apps/web	로그인·대시보드·로그/AI 표시
apps/api	인증·회원가입·로그 조회·AI API
services/builder	checkout·Docker·LIB/RUN 등록

REQUEST & DATA FLOW

로컬 PC push부터 화면 표시까지

GIT PUSH PIPELINE

- 01 로컬 PC
- git push http(s)://<server>:8080/git/{team}/{user}.git
- 02 Nginx 인증
- auth_request → FastAPI /internal/git-auth
- Basic Auth 확인 → users 조회 → team/user 저장소 권한 확인
- 03 Git 저장
- 인증 성공 요청만 fcgiwrap의 git-http-backend로 전달
- pack과 ref를 /repos/{team}/{user}.git에 기록
- 04 Hook 실행
- bare repo의 post-receive
- /app/logplus-platform/services/builder/build.sh users_id team_id
- 05 Build / Run
- 작업본 fetch → reset --hard origin/master → clean -fd
- Dockerfile 사용 또는 언어별 Dockerfile 생성
- docker build → docker run --rm
- 06 로그 등록
- 설치·빌드 출력 *_LIB.log
- 프로그램 실행 출력 *_RUN.log
- FastAPI POST /logs를 lib/run 각각 호출
- MariaDB build_logs에 실제 파일 경로와 메타데이터 저장

WEB → SCREEN PIPELINE

- 07 로그인
- POST /user/login
- 응답 users_id, team_id를 Zustand 메모리에 저장
- 08 로그 목록
- POST /db/getLog { users_id, team_id }
- build_log_id, log_type, created_at 목록 반환
- 09 로그 본문
- POST /db/getLog { ..., build_log_id }
- DB의 log_path를 읽어 실제 파일 본문 반환
- 10 React 렌더링
- selectedLog.log_content를 줄 단위로 표시
- ERROR / WARN / INFO / DEBUG 레벨 강조
- 11 AI 선택 분석
- POST /ai/reanalyze
- 로그 전처리 → Ollama Chat Completions → 분석 결과 생성
- analyses/*_AI_{analysis_id}.txt와 ai_analyses에 저장

저장 경계와 검증 상태

MARIADB

teams	팀 식별자
users	로그인 ID, team_id, role
build_logs	로그 ID, 유형, 실제 파일 경로, 생성 시각
ai_analyses	분석 ID, 대상 로그, 분석 파일 경로, 생성 시각

FILE SYSTEM

```
/repos/{team}/{user}.git
/app/logplus-platform/projects/{team}/{user}
/app/logplus-platform/logs/{team}/{user}
├─ *_LIB.log
├─ *_RUN.log
└─ analyses/*_AI_{analysis_id}.txt
```

확인된 근거

- [OK] GET /health에서 API와 DB 연결 정상 응답 기록
- [OK] 무인증 Git 요청 401, 잘못된 경로 403, 인증 성공 204 기록
- [OK] builder의 checkout → Docker build 성공 → run → /logs 2회 기록
- [OK] /db/getLog 200 및 /ai/reanalyze 200 기록
- [OK] /logs/test1에서 test_project/test1 LIB/RUN 로그 목록 확인

현재 구현에서 실제 연결된 화면

로그인, 로그 목록, 로그 상세 본문, AI 재분석 결과

주요 API

```
/user/login · /user/register · /logs
/db/getLog · /ai/reanalyze · /internal/git-auth
```

아직 샘플 데이터인 화면 영역

통계 카드, 최근 알림, 예외 TOP, 막대 차트

남은 운영 과제

1. Vite 5174 단일 프로세스와 API/Nginx 실행 상태 재확인
2. 기존 프론트 TypeScript build 오류 정리
3. 세션/토큰 인증과 비밀번호 해시 적용
4. CORS 제한과 DB 비밀번호 환경변수화
5. builder queue/lock 및 실제 통계 집계 API 추가

최종 상태 메모

문서 작업 중간에는 API health와 Nginx 인증 게이트가 응답했지만, 최종 probe에서는 API 8000과 Nginx 8080이 connection refused였다. 서비스 재기동은 하지 않았으며, 다음 작업에서 실행 상태를 재확인한다.